

A Major Project Report

On

**An End-to-End Deep Learning Regressor for Predicting Stress
Levels from Physiological Signals**

Submitted

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

By

MOTHE. ROHITH

21641A05Q6

Under the Guidance of

Mrs.P.Vemulamma

Assistant Professor



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING VAAGDEVI
COLLEGE OF ENGINEERING**

(UGC Autonomous, Accredited by NBA, Accredited by NAAC with “A”)

Warangal – 506001

(2021-2025)

VAAGDEVI COLLEGE OF ENGINEERING

(Autonomous)

(UGC Autonomous, Accredited by NAAC with “A”)



Bollikunta, Khila Warangal (Mandal), Warangal Urban – 506005(T.S)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project entitled “ **An End-to-End Deep Learning Regressor for Predicting Stress Levels from Physiological Signals**” is submitted by **M.Rohith** bearing Hall Ticket Nos.**21641A05Q6** in partial fulfillment of the requirements for the award of the Degree in Bachelor of Technology in Computer Science and Engineering during the academic year 2024-2025.

GUIDE

Mrs.P.Vemulamma

HEAD OF THE DEPARTMENT

Dr.N.Satyavathi

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to take this opportunity to express our sincere gratitude and deep respect to **Dr. K. Prakash, Principal of Vaagdevi College of Engineering**, for providing us with all the necessary assistance and for his support and inspiration, which enabled us to carry out this project work at the institute.

Our heartfelt thanks extend to **Dr. N. Satyavathi, Associate Professor, Head of the Department**, Computer Science and Engineering, Vaagdevi College of Engineering, for equipping us with the required infrastructure and granting us the freedom to successfully conduct this project work.

We are immensely grateful to our guide, **Mrs.P.Vemulamma, Assistant Professor**, Department of Computer Science and Engineering, for his/her constant support and invaluable guidance throughout the duration of this project work. Their insights and encouragement have been instrumental in our progress.

We would also like to thank the Project Coordinators, **Dr.K.Rekha Devi, Assistant Professor** and **Mr. Ch. Aravind Kumar, Assistant Professor**, for their thoughtful suggestions, motivation, and encouragement, which contributed significantly to the completion of our project.

Finally, we extend our sincere thanks to all the teaching and non-teaching staff members of the Department of Computer Science and Engineering, whose direct and indirect support was vital in ensuring the successful completion of this work.

MOTHE. ROHITH (21641A05Q6)

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “ **AN END**

TO END DEEP LEARNING REGRESSOR FOR PREDICTING STRESS LEVELS FROM PHYSIOLOGICAL SIGNALS” in the partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** and submitted in the **DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING, VAAGDEVI COLLEGE OF ENGINEERING**, is an authentic record of my own work carried out during the time period from 2021 to 2025 under the Guidance of **Mrs.P.Vemulamma, Assistant Professor**. The matter presented in this project report has not been submitted by me for the award of any other degree of this or any other institute.

M. ROHITH

21641A05Q6

ABSTRACT

Stress monitoring via physiological signals has the potential to transform healthcare by enabling objective, continuous assessment, yet existing workflows remain fragmented relying on episodic questionnaires, manual scoring, and separate scripts for machine-learning. To address this gap, we developed an end-to-end desktop application with a Tkinter graphical user interface that guides users through data ingestion, preprocessing, model training, evaluation, and on-demand prediction without requiring code-level interaction. Upon uploading a CSV of physiological measurements, the system displays schema information, summary statistics, missing-value counts, and applies z-score normalization to all feature columns. A correlation heatmap facilitates exploration of inter-feature relationships. Two regression pipelines were implemented: a non-parametric K-Nearest Neighbors (KNN) regressor and a hybrid Multi-Layer Perceptron (MLP)–driven Linear Regression model. Models persisted as serialized artifacts to avoid redundant retraining. On a held-out 20 % test set, the KNN regressor achieved a Mean Absolute Error (MAE) of 0.4867, Mean Squared Error (MSE) of 0.3717, Root Mean Squared Error

(RMSE) of 0.6097, and R^2 of 0.9241. The hybrid pipeline improved accuracy—MAE = 0.3903,

MSE = 0.2389, RMSE = 0.4888, $R^2 = 0.9528$ —demonstrating relative error reductions of ~19.8 % (MAE/RMSE) and a ~3.1 % increase in explained variance. Interactive scatter plots of predicted vs. actual values, along with textual performance summaries, provide transparent model assessment. Finally, users can upload new data for immediate inference, with predictions appended and displayed alongside raw inputs. This integrated, click-driven tool democratizes access to advanced stress-level modeling, offering both researchers and practitioners a reproducible, visually guided environment to develop, compare, and deploy regression models on physiological datasets.

INDEX PAGE

CHAPTER	TITLE	PAGE NO.
1.	INTRODUCTION	01-03
2.	LITERATURE SURVEY	04-06
3.	EXISTING SYSTEM	07-08
4.	PROPOSED SYSTEM	09-16
5.	UML DIAGRAM	17-24
6.	DESIGN OF MINI PROJECT	25-30
6.1	SOFTWARE REQUIREMENTS	25-28
6.2	HARDWARE REQUIREMENTS	29-30
7.	REQUIREMENTS & SYSTEM STUDY	31-35
7.1	FUNCTIONAL REQUIREMENTS	31-32
	7 . 2 SYSTEM STUDY	33-34

8. SOURCE CODE 35-44

9. RESULTS 45-53

10. CONCLUSION & FUTURE WORK 54

11. REFERENCES 55-56

FIGURES INDEX

Figure No.		Page No.
Fig: 1.1	Existing System	
Fig:2.2	Proposed Methodology	
Fig: 3.1	KNN Regressor	
Fig: 4.1	UML Diagram	18
Fig: 4.2	Use Case Diagram	19
Fig: 4.3	Data Flow Diagram	20
Fig: 4.4	Deployment Diagram	21
Fig: 4.5	Component Diagram	22
Fig: 4.6	Collaboration Diagram	23
Fig: 5.1	Sequence Diagram	24
Fig: 5.2	Deployment diagram.	25
Fig: 5.3	Sequence Diagram	26
Fig: 5.4	Activity diagram.	28
Fig: 5.5	The class diagram	29
Fig: 9.1	Web application of proposed AI-powered skin cancer prediction system.	49
Fig: 9.2	User signup page.	49
Fig: 9.3	Successful signup completion page.	50
Fig: 9.4	User login page.	51
Fig: 9.5	Showing navbar tasks build and train deep CNN	52
Fig: 9.6	Confusion matrix obtained using proposed deep CNN model.	53
Fig: 9.7	Web application displaying the performance metrics of deep CNN model.	53
Fig: 9.8	Console showing the layered architecture of proposed deep CNN model.	54

CHAPTER 1 INTRODUCTION

1.1 Overview

In the fast-paced modern world, stress has emerged as a pervasive and significant health concern, affecting individuals across all walks of life. The World Health Organization has declared stress a global epidemic, with its impacts ranging from decreased productivity and quality of life to severe physical and mental health issues. As awareness of these detrimental

effects grows, so does the need for accurate, real-time stress detection methods that can facilitate timely interventions and support effective stress management strategies.

Traditional approaches to stress assessment have largely relied on self-reports and occasional clinical evaluations. However, these methods are limited by their subjective nature, infrequency, and inability to capture real-time stress fluctuations. The advent of wearable technology has opened new avenues for continuous, objective stress monitoring through the measurement of various physiological signals [1, 2]. These devices can capture a wealth of data, including heart rate variability, electrodermal activity, skin temperature, and accelerometer data, providing a more comprehensive picture of an individual’s physiological state.

Despite this technological advancement, the challenge of accurately interpreting these multi-modal physiological signals to detect stress remains significant. Early attempts at physiological stress detection often focused on single-modal approaches, utilizing individual biomarkers such as heart rate or skin conductance. While these methods showed promise, they failed to capture the complex, multi-faceted nature of the human stress response [3, 4]. More recent studies have explored multi-modal approaches, combining data from various physiological signals to improve detection accuracy. However, many of these methods still rely heavily on time-domain features, potentially overlooking valuable information contained in the frequency domain of these signals [5].

1.2 Research Motivation

Chronic stress is a pervasive contributor to a wide array of physical and mental health issues, including cardiovascular disease, anxiety disorders, and burnout. Traditional stress assessment methods—such as self-report questionnaires—are episodic and subjective, failing to capture rapid fluctuations or provide objective, continuous monitoring. Advances in wearable sensors now allow real-time collection of physiological signals (e.g., heart rate variability, skin conductance), but there remains a gap in accessible, end-to-end tools that translate these raw data into actionable stress-level predictions. This project seeks to bridge that gap by integrating data handling, model training, evaluation, and deployment within a single, user-friendly application.

1.3 Problem Definition

Existing clinical and research workflows rely heavily on manual scoring of questionnaire responses, which introduces recall bias and limits assessment frequency to scheduled visits.

Moreover, standalone scripts for machine-learning model development require programming expertise and often lack visualization or persistence layers, making them inaccessible to many practitioners. There is a clear need for a cohesive system that automates data preprocessing, model training, performance evaluation, and prediction—without demanding code-level interaction—while ensuring reproducibility and ease of use.

1.4 Research Objectives

- Design & implement a graphical user interface that guides users through each step of the stress-prediction pipeline, from data ingestion to model deployment.
- Develop & compare two regression approaches—a non-parametric KNN regressor and a hybrid MLP-driven linear regression—to establish performance baselines.
- Automate & visualize key processes, including exploratory data analysis, correlation mapping, and “predicted vs. actual” plotting, to enhance interpretability.
- Persist & reuse trained models via serialization, enabling rapid re-evaluation and prediction on new datasets without redundant retraining.

1.5 Significance

By consolidating the entire machine-learning workflow into a single application, this methodology democratizes stress monitoring, making it accessible to clinicians, researchers, and even end users without programming backgrounds. The hybrid modeling strategy leverages both nonlinear feature learning and linear interpretability, often yielding higher accuracy than either approach alone. Real-time visualization and persistent model storage accelerate iterative refinement, while standardized performance metrics foster transparency and reproducibility—critical for both clinical adoption and research validation.

1.6 Applications

- **Clinical Psychology & Psychiatry:** Continuous monitoring of patient stress levels between appointments to inform treatment adjustments.
- **Workplace Well-being Programs:** Deployable on office workstations or wellness kiosks to assess employee stress trends and trigger timely interventions.
- **Sports & Performance Optimization:** Real-time feedback on athlete stress during training, enabling coaches to balance exertion and recovery.

- **Telehealth & Remote Monitoring:** Integration with wearable devices to provide remote practitioners with objective stress metrics alongside other vital signs.

CHAPTER 2

LITERATURE SURVEY

Wearable devices coupled with machine learning techniques have emerged as powerful tools for stress detection, offering continuous, non-invasive monitoring capabilities in real-world environments. A comprehensive review highlighted the significance of physiological indicators, including heart rate variability (HRV), skin temperature, and EDA in stress detection [3]. This work emphasized the crucial role of both time-domain and frequency-domain analyses for precise stress monitoring. However, existing studies often focus on either time-domain or frequency-domain features separately, limiting their ability to fully capture stress-related physiological variations. Subsequently, a systematic review presented generalizable machine learning models for stress monitoring, addressing critical challenges such as dataset transferability and model robustness across diverse populations [6]. While these models improve generalizability, they often overlook the challenges posed by intermittent data collection in real-world occupational settings.

Recent advances in predictive modeling have demonstrated the effectiveness of integrating multiple data sources. Comparative studies examining various stress prediction models that combine smartwatch physiological signals with self-reported measures revealed enhanced predictive performance through this dual-source approach [7]. Nevertheless, reliance on self-reported data introduces subjectivity, which may affect model reliability and applicability in real-time monitoring. In parallel, research introduced an explainable deep learning framework for stress detection using wearable sensor data, providing crucial transparency in model interpretation for healthcare applications [8]. Although explainability improves trust in deep learning models, further enhancements are needed to balance interpretability with predictive accuracy. Furthermore, investigations into autoencoder-based approaches demonstrated the effectiveness of temporal feature extraction from wearables for forecasting both stress and mood, highlighting the potential of unsupervised learning methods in personalized health monitoring [9]. Despite their success, autoencoder-based methods often require extensive tuning and may struggle with diverse physiological patterns in occupational stress scenarios.

Recent sensor-based methods have advanced stress detection by integrating new data modalities. For example, magnetostrictive polymer composites (MPCs) using UV-curable

epoxy resin demonstrated reliable stress detection through changes in magnetic flux, offering potential to refine stress monitoring systems by augmenting time and frequency domain features [10]. While this approach showcases novel sensor technology, its practicality for widespread wearable integration remains uncertain.

Furthermore, deep learning advancements in sensor-based recognition have enabled automatic feature extraction across complex physiological signals, addressing challenges such as unsupervised and incremental learning. These frameworks improve adaptability and interpretability, enhancing stress detection in varied real-world contexts [11]. However, many existing models lack mechanisms to effectively integrate multi-modal data, limiting their ability to capture stress responses comprehensively.

The role of specific physiological parameters in stress detection has been extensively investigated. Novel methods for mental stress assessment using HRV derived from electrocardiogram (ECG) signals demonstrated high precision in stress quantification [12]. Despite their accuracy, ECG-based approaches often require specialized sensors, reducing feasibility for daily wear. Additionally, pilot studies contributed to the field through the introduction of the Stress-Predict dataset, establishing a robust foundation for developing and validating stress prediction algorithms across diverse conditions [13]. While valuable for benchmarking, these datasets may not fully represent stress variability in high-intensity professional settings. Research into the feasibility of combining wearable and self-reported measures in controlled lab environments has illuminated both the potential and limitations of deploying these techniques in real-world applications [14]. Yet, stress assessment in controlled environments may not directly translate to occupational settings where intermittent data collection is a major challenge.

In professional environments, research explored embedded devices for continuous stress monitoring, providing valuable insights into wearable adaptation for demanding workplace settings [15]. However, many existing workplace monitoring solutions require high data availability, which is not always feasible in dynamic job roles such as nursing. These findings suggest practical applications for occupational health programs. Complementing this work, investigations into EEG-based brain-computer interfaces for stress detection presented an innovative approach that combines neural indicators with physiological data for comprehensive stress assessment [16]. Despite their novelty, EEG-based systems are often intrusive and less practical for long-term stress tracking in daily occupational settings. Real-time prediction models designed for integrating wearable devices into daily life further highlight the practical

aspects of these systems [17]. Nevertheless, most real-time models struggle with handling missing or intermittently collected data, a crucial issue in professional environments.

Recent research has increasingly focused on personalization in stress monitoring solutions. Extensive investigations into wearable-based stress detection in semi-controlled settings identified both opportunities and limitations of current technology [12]. However, achieving a balance between generalization and personalization remains a challenge in real-world applications. Furthermore, studies proposed generalizable machine learning approaches addressing feature extraction and model generalization across various contexts, enhancing the versatility of stress monitoring systems [6]. Yet, many approaches still struggle with effectively integrating frequency-domain features, which are essential for capturing stress-related signal variations. Additional research focused on leveraging bio-signals for personalized stress detection, demonstrating the efficacy of individual physiological patterns for enhancing predictive accuracy [18]. However, ensuring model adaptability across different individuals and work environments remains an open problem. Recent developments in real-time physiological data analysis have further advanced personalized stress detection models, facilitating both immediate interventions and longitudinal stress tracking (Ceren Ates et al., 2024). Despite these advances, a unified framework that effectively integrates multi-modal signals for stress detection under real-world intermittent data conditions is still lacking.

CHAPTER 3

EXISTING SYSTEM

In the conventional approach, stress levels are gauged via paper-based or digital self-report questionnaires (e.g., Perceived Stress Scale) filled out at clinic visits. A clinician or researcher manually scores each item, interprets the cumulative score against predefined thresholds, and then provides feedback or intervention recommendations. Data storage is typically in static files or spreadsheets, and analysis requires separate statistical software. This workflow is inherently episodic—relying on patient recall and clinician availability—and lacks real-time monitoring or automated insight generation.

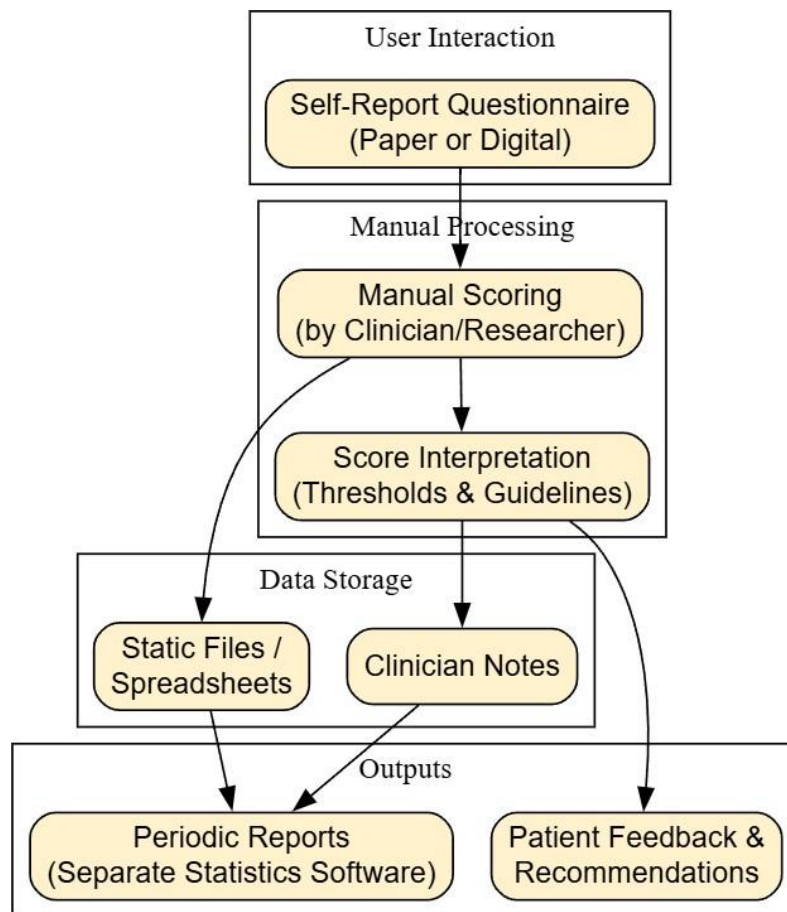


Fig. 3.1: Traditional system architecture.

Limitations

- **Subjectivity & Recall Bias:** Relies on user self-report and memory, leading to inconsistent or inaccurate data.
- **Time-Intensive:** Manual scoring and interpretation by clinicians slows down feedback loops.
- **Poor Scalability:** As the number of users grows, managing and analyzing paper forms or spreadsheets becomes unwieldy.
- **Lack of Real-Time Insights:** No continuous or automated monitoring; stress events between assessments go unnoticed.
- **Data Fragmentation:** Questionnaire data, clinician notes, and statistical analyses live in separate silos, making longitudinal tracking cumbersome.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 Overview

The proposed methodology begins with a user-friendly Tkinter GUI that orchestrates each step of the machine-learning pipeline. First, raw physiological data are ingested via a file-upload interface and loaded into a pandas DataFrame. Next, a preprocessing module handles data cleaning, normalization, and exploratory analysis—producing summary statistics and a correlation heatmap. Two regression pipelines are then available: a KNN regressor and a hybrid MLP-driven linear regression, each trained (or loaded) and evaluated on hold-out data. Finally, the trained models persisted to disk and can be applied to new datasets, with performance metrics and prediction results visualized directly in the app.

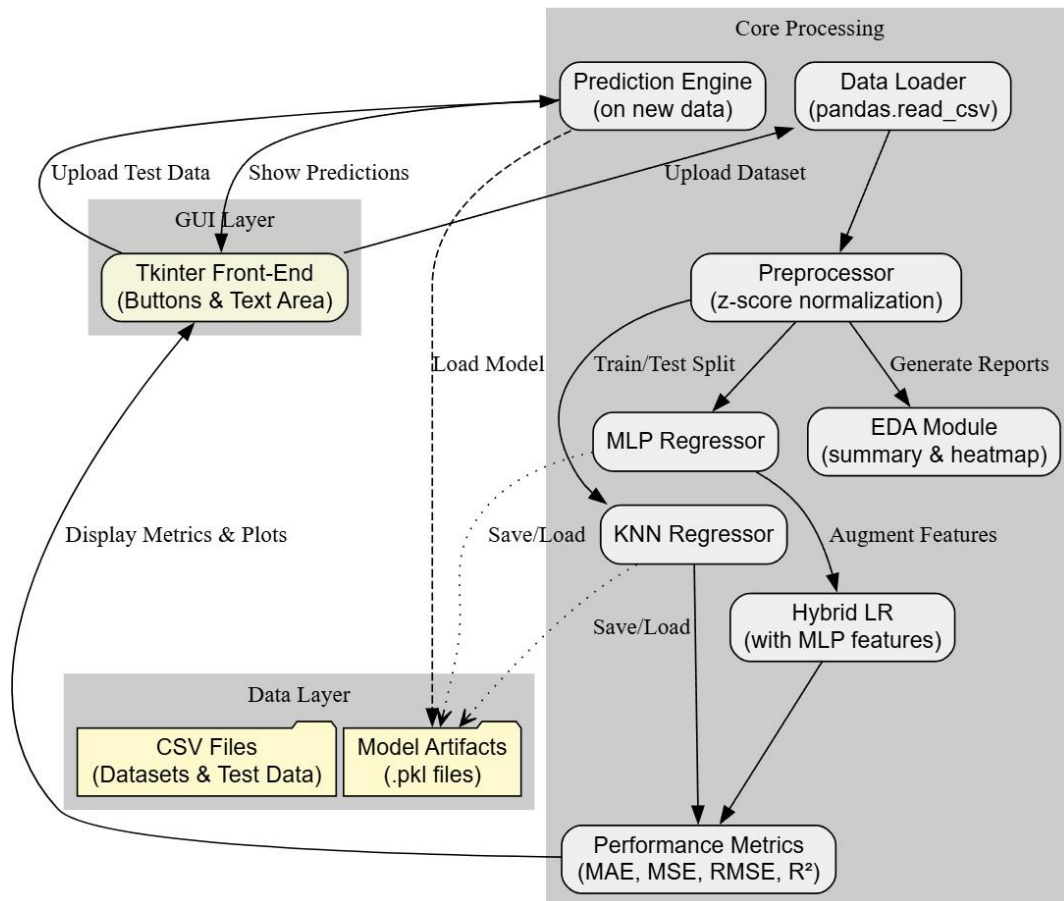


Fig. 4.1: Proposed system architecture of MLP-driven LR for prediction of stress from physiological signals data.

Here's the stepwise explanation:

Step 1. User-Friendly GUI Front End

- Built with Tkinter, the app presents a window with clearly labeled buttons (“Upload Stress Dataset,” “Data Preprocessing and EDA,” “KNN Regressor,” “Hybrid MLP-Driven LR,” “Prediction on Test Data,” and “Close Application”).
- All outputs (logs, tables, metrics) appear in a scrollable text panel, and plots pop up in Matplotlib windows, so you never have to touch the console.

Step 2. Data Loading & Inspection

- “Upload Stress Dataset” lets you pick a CSV file of physiological measurements. The code reads it into a pandas DataFrame and displays the path and initial rows.
- This makes it trivial to swap in different datasets without rewriting any code.

Step 3. Preprocessing & Exploratory Data Analysis

- “Data Preprocessing and EDA” computes and shows summary statistics, missing-value counts, and unique-value counts.
- It then splits the data into training and testing subsets (80/20), z-scores the feature columns, and renders a full correlation heatmap so you can spot which signals tend to move together.

Step 4. Model Training & Evaluation

- **KNN Regressor:** On demand, the app either loads a saved `KNeighborsRegressor` or fits a new one to the training data, then reports MAE, MSE, RMSE, and R^2 along with a “Predicted vs. Actual” scatter plot.
- **Hybrid MLP-Driven Linear Regression:** First, an `MLPRegressor` is trained (or loaded), whose own predictions are appended as a new feature. A `LinearRegression` is then fit on this augmented data to create a hybrid pipeline—again evaluated by the same four metrics and visualized.

Step 5. Making Predictions on New Data

- “Prediction on Test Data” opens another file dialog so you can feed in unseen physiological records. Those records are passed through the already-trained MLP to yield stress-level predictions, which are displayed alongside the input data in the GUI.

Step 6. Persistence & Reusability

- Both models are saved to disk (.pkl files) after training, so repeated runs of the app skip retraining unless you deliberately delete or rename those files.
- This makes experimentation fast: you can tweak parameters, retrain once, and then instantly re-evaluate or predict.

4.2 Preprocessing and Normalization

1. Data Inspection & Cleaning

- After loading the CSV into `df1`, the code prints and displays:

- DataFrame schema (`df1.info()`)
- Summary statistics (`df1.describe().T`)
- Null-value counts (`df1.isnull().sum()`)
- Unique-value counts (`df1.nunique()`)

- This gives you an immediate sense of column types, missing data, and the distribution of each variable before any transformation.

2. Feature/Target Separation and Train–Test Split

- The target variable is extracted as
`y = df1['target'].values`
- All remaining columns become features:

`X = df1.drop(columns=['target'], axis=1).values`

- An 80/20 split is performed with a fixed random state for reproducibility:

`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=77)`

Normalization (Standardization)

- Rather than using scikit-learn’s built-in scalers in this step, the code applies z-score normalization directly to each feature column in the original DataFrame:

for idx in range(len(df1.columns) - 3): # excluding presumed target-related columns

col = df1.columns[idx]

df1[col] = (df1[col] - df1[col].mean()) / df1[col].std()

- For each feature column, subtract its mean and divide by its standard deviation. ○ After this transform, each column has a mean of 0 and a standard deviation of 1, which helps models converge faster and prevents features with larger scales from dominating.

Although a `MinMaxScaler(0,1)` is instantiated at the top of the script, it isn't actually invoked in the preprocessing routine—so all scaling in the current workflow is via manual z-scoring.

4.3 Model Building and Training

The internal workings of each regression model can be visualized as a sequential data-flow pipeline. For the KNN regressor, the core steps involve computing distances between a query point and all stored training samples, selecting the nearest neighbors, and averaging their target values to make a prediction. In the hybrid MLP-driven LR pipeline, the network first uses a feedforward MLP to produce an auxiliary prediction, then augments the original feature vector with this MLP output before passing everything through a linear regression layer.

4.3.1 KNN Regressor

The K-Nearest Neighbors regressor is a simple, instance-based learning method that makes predictions by averaging the target values of the K most similar training samples. It requires no explicit training phase—just storing the feature vectors and associated targets—then computing distances (typically Euclidean) at inference time. Because it relies on local neighborhood structure, KNN naturally adapts to arbitrary data distributions but can become slow on large datasets. Its non-parametric nature means it makes minimal assumptions about underlying data relationships, making it a robust baseline for stress-level estimation.

Explanation:

1. **Input Feature Vector:** The new sample's features are fed into the KNN pipeline.
2. **Distance Calculation:** Euclidean distances (or another metric) are computed between this query vector and every stored training sample.
3. **Neighbor Selection:** The K training samples with the smallest distances are chosen as

“nearest neighbors.”

4. **Aggregation:** The regressor simply takes the mean of those neighbors’ target (stress) values.
5. **Output:** That average becomes the final prediction for the input sample.

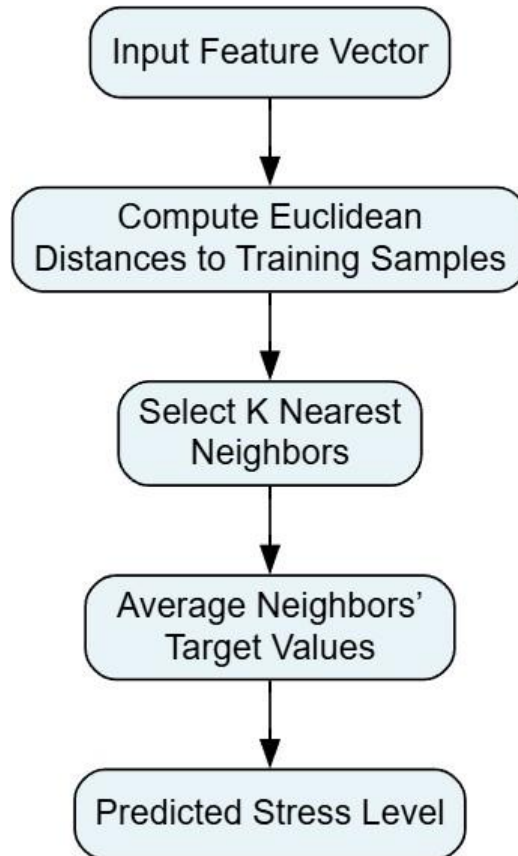


Fig. 4.2: KNN regressor internal operation.

4.3.2 Hybrid MLP-driven LR

This two-stage pipeline first passes input features through a multi-layer perceptron (MLP) to capture nonlinear patterns in physiological signals. The MLP’s scalar output is then concatenated with the original feature vector and fed into a linear regression model. By blending the MLP’s capacity for complex feature learning with the interpretability and efficiency of linear regression, the hybrid approach often achieves enhanced accuracy while maintaining computational efficiency at inference time.

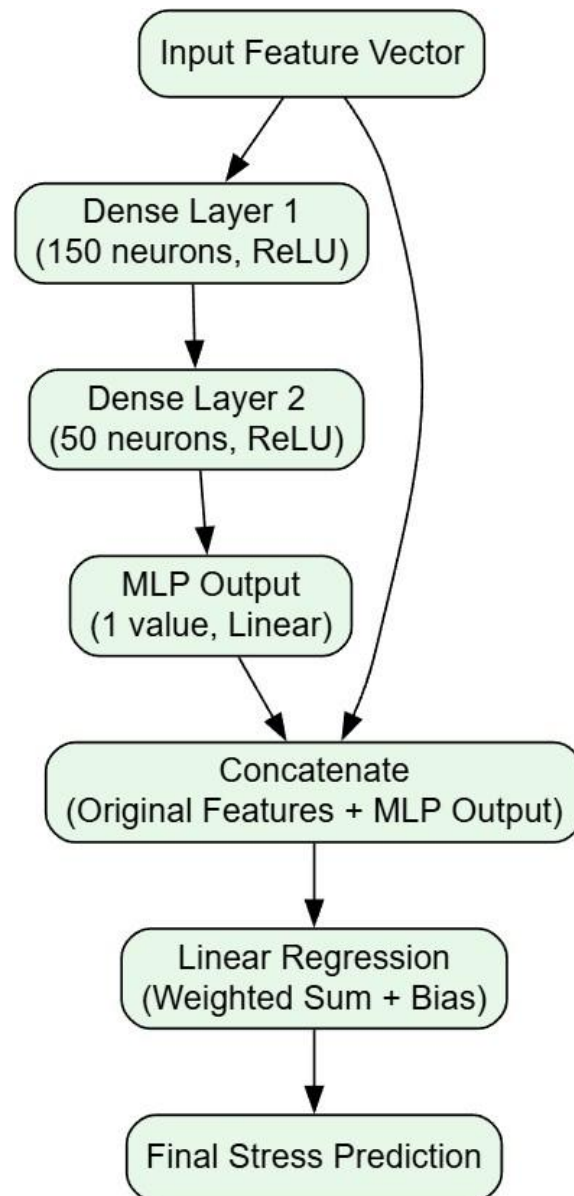


Fig. 4.3: Internal workflow of MLP-driven LR model.

Explanation:

1. **Input Feature Vector:** The same preprocessed features enter two branches.
2. **MLP Regressor:** A multi-layer perceptron with hidden layers of sizes 150 and 50 performs a forward pass, outputting a preliminary stress estimate.
3. **Feature Augmentation:** That single MLP output is appended to the original feature vector, creating an extended input.
4. **Linear Regression:** A simple weighted sum (plus intercept) is fit on this augmented data, leveraging both the original signals and the MLP's learned nonlinearity.
5. **Final Prediction:** The LR's output is taken as the refined stress-level prediction.

4.4 Advantages

1. **End-to-End, Click-Driven Workflow:** All stages—from data ingestion and cleaning through model training, evaluation, and prediction—are accessible via a single GUI. This eliminates the need for command-line interaction and makes the process accessible to non-programmers.
2. **Modular, Extensible Architecture:** The clear separation between the GUI layer, preprocessing/EDA, individual model modules (KNN and MLP), and the prediction engine means you can readily swap in new algorithms or add additional analytics without reworking the entire codebase.
3. **Hybrid Modeling for Enhanced Accuracy:** By combining an MLP’s capacity to learn complex, nonlinear relationships with the transparency and efficiency of linear regression, the pipeline captures rich feature interactions while keeping inference fast and interpretable.
4. **Built-in Model Persistence:** Trained models are saved to disk as .pkl artifacts on first run and automatically reloaded on subsequent launches. This avoids repeated retraining and accelerates experimentation cycles.
5. **Integrated Visualization & Reporting:** Key insights—summary statistics, null-value checks, a correlation heatmap, and “predicted vs. actual” scatter plots—are generated on demand. Users gain immediate, graphical feedback on data quality and model performance.
6. **Flexible, Comparative Baseline:** Including both a simple, non-parametric KNN regressor and the more advanced hybrid pipeline provides a built-in benchmark. Users can directly compare model behaviors, selecting the best balance of accuracy, speed, and interpretability for their needs.

CHAPTER 5

UML DIAGRAMS

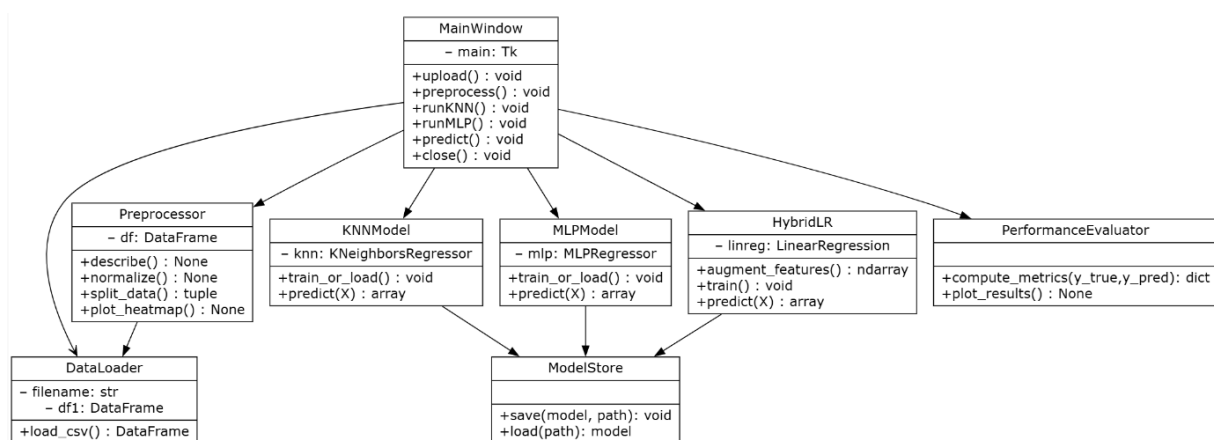
Unified Modeling Language (UML) is a standardized visual language used to model, design, and document the architecture of software systems. It provides a set of graphical notations to represent the structure and behavior of a system, making complex systems easier to understand and communicate among developers, stakeholders, and business analysts.

Key Points About UML

- **Standardized Notation:** UML offers a universal set of symbols and diagrams that standardize how software systems are described, which helps teams speak the same language regardless of their background or the programming language they use.
- **Types of Diagrams:** UML includes various diagrams that can be categorized into two main types:
 - **Structural Diagrams:** These describe the static aspects of the system. Examples include Class Diagrams, Component Diagrams, and Deployment Diagrams.
 - **Behavioral Diagrams:** These focus on the dynamic aspects and interactions within the system. Examples include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and Collaboration Diagrams.
- **System Documentation and Communication:** UML serves as an effective tool for documenting system requirements, design decisions, and the overall architecture. It helps bridge the gap between technical and non-technical stakeholders by providing clear, visual representations of the system.
- **Design and Analysis:** By modeling different aspects of a system, UML enables developers to analyze and validate the design early in the development process. This can lead to better decision-making, reduced complexity, and improved system maintainability.
- **Flexibility:** UML is versatile and can be used across a wide range of applications, from small-scale projects to large, complex systems. It supports object-oriented design principles and can be adapted to various methodologies such as Agile or Waterfall.

Class Diagram

The class diagram captures the static structure of the stress-prediction application, showing its primary classes, their attributes, methods, and the relationships between them. It highlights how the GUI layer connects to data-handling and model classes, and how model persistence and evaluation are encapsulated. By visualizing classes like `MainWindow`, `DataLoader`, `Preprocessor`, and the various model classes, we get a clear picture of the system's modular design. This diagram forms the blueprint for coding and refactoring, ensuring each responsibility resides in its own class. It also helps identify dependencies and potential extension points for future features.



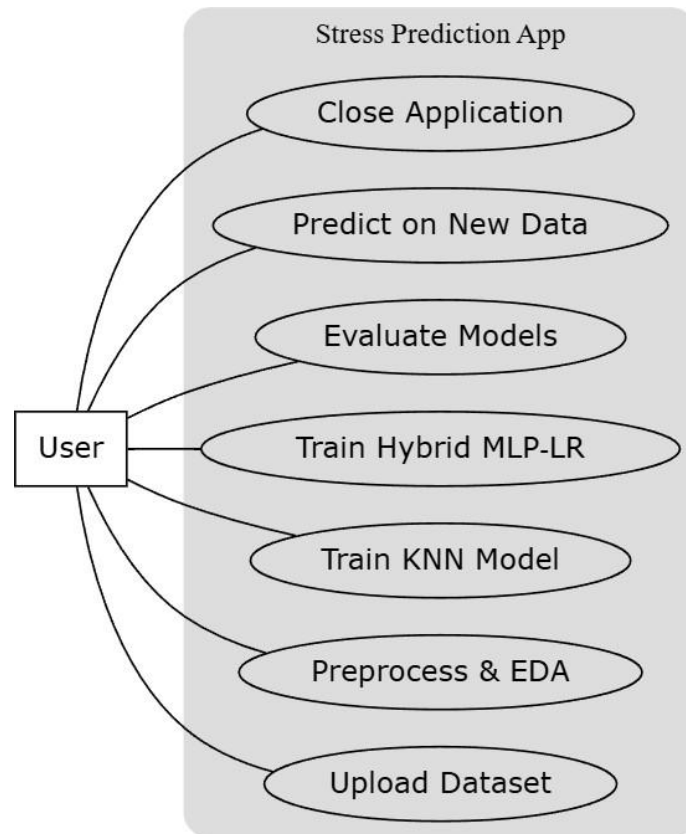
Each rectangle represents a class with its private attributes (–) and public methods (+).

- `MainWindow` orchestrates user interactions and delegates to other classes.
- `DataLoader` encapsulates file-loading logic using pandas.
- `Preprocessor` handles EDA tasks (description, normalization, splitting, heatmap).
- `KNNModel`, `MLPModel`, and `HybridLR` implement the two regression pipelines, each managing its own training or loading logic.
- `PerformanceEvaluator` centralizes metric computation and plotting.
- `ModelStore` abstracts persistence via joblib. Arrows indicate which classes call or depend on others, illustrating clear separation of concerns.

Use Case Diagram

The use case diagram outlines the interactions between the primary actor—the end user—and the stress-prediction system. It shows the system boundary and the key functions exposed via

the GUI: uploading a dataset, preprocessing and exploring it, training each model, evaluating performance, making predictions on new data, and closing the application. This high-level view helps stakeholders understand what services the system provides and who initiates them. It's an effective tool for requirement gathering and validating that all necessary capabilities are covered.



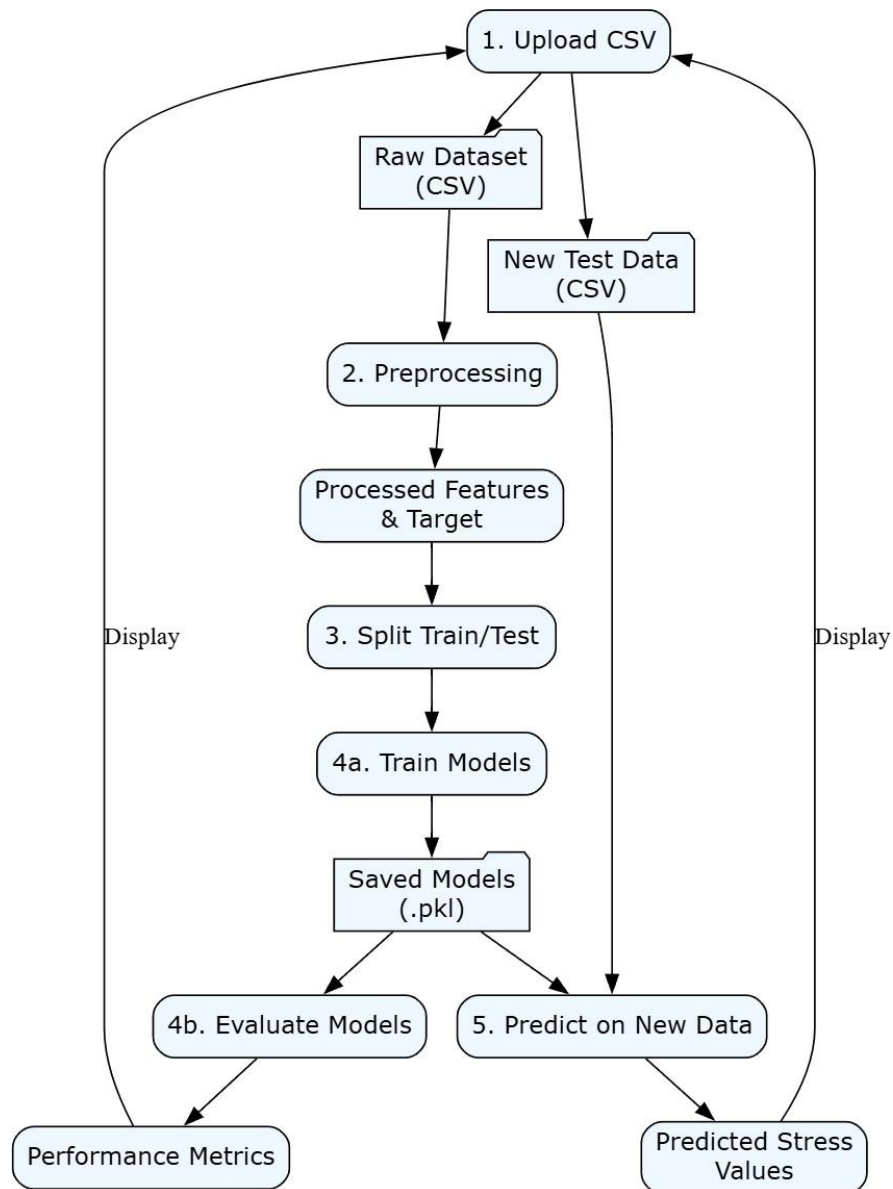
Explanation

- Actor: The User represents anyone operating the GUI.
- Use Cases: Each oval is a core function provided by the application.
- System Boundary: The rounded rectangle groups all use cases within the “Stress Prediction App.” Lines connecting User to each use case indicate the user’s initiation of that function. This diagram ensures the system meets all user-triggered requirements.

Data Flow Diagram

The data flow diagram (DFD) Level 1 illustrates how data moves through the system’s major processes, from user input to data storage and visualization. It captures the transformation pipelines—uploading raw CSVs, preprocessing, model training, evaluation, and prediction—

and shows where data is stored (both transiently in memory and persistently on disk). This diagram is useful for understanding bottlenecks, data dependencies, and for designing interfaces between modules.



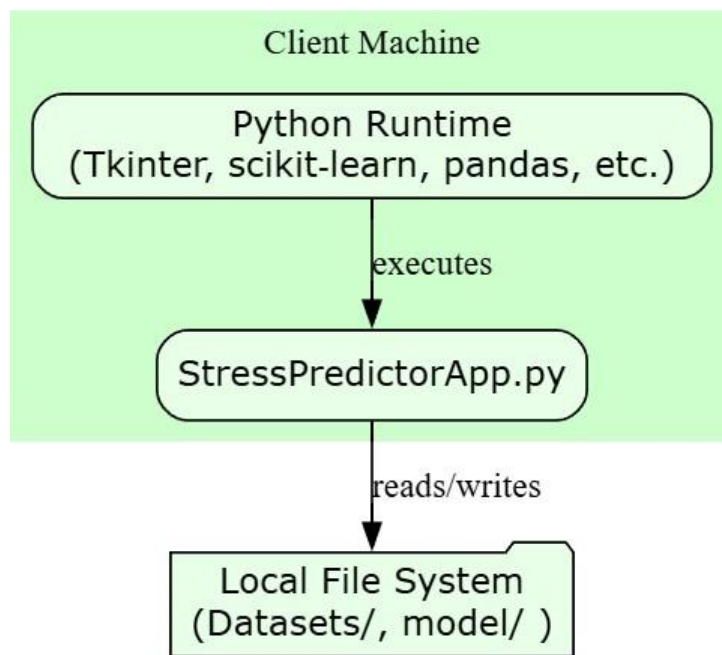
Explanation

1. Upload CSV → Raw Data: The user selects a dataset.
2. Preprocessing: Cleans, describes, and normalizes data, producing a feature matrix and target vector.
3. Train/Test Split: Divides data for modeling.
4. Model Training & Evaluation: Trained models are saved; evaluation yields metrics.
5. Prediction: New data is run through persisted models to generate predictions.

Data stores (.csv, .pkl) and transformations are clearly mapped.

Deployment Diagram

The deployment diagram shows how software artifacts are allocated to physical nodes in the target environment. Since this is a desktop application, the primary node is the user's local machine running Python with required libraries. It also illustrates where models and data files reside on the file system. This view assists IT teams in planning installation, dependencies, and resource allocation.



Explanation

- Client Machine: The user's desktop with Python installed.
- Python Runtime: Hosts all dependencies (Tkinter, pandas, scikit-learn, Matplotlib, etc.).
- StressPredictorApp.py: The main application script.
- Local File System: Houses input datasets and serialized model files.

Arrows indicate execution and file I/O interactions, clarifying deployment steps.

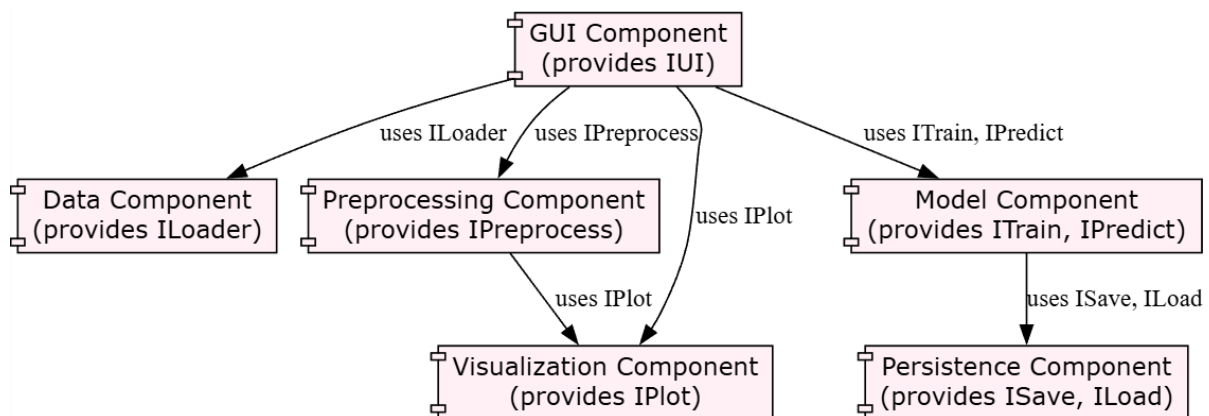
Component Diagram

The component diagram decomposes the system into logical components and their interfaces, showing how the GUI, data-handling, processing, modeling, persistence, and visualization pieces fit together. It highlights provided and required interfaces, helping developers

understand how to integrate or replace components. This diagram is especially valuable for large teams working on different subsystems.

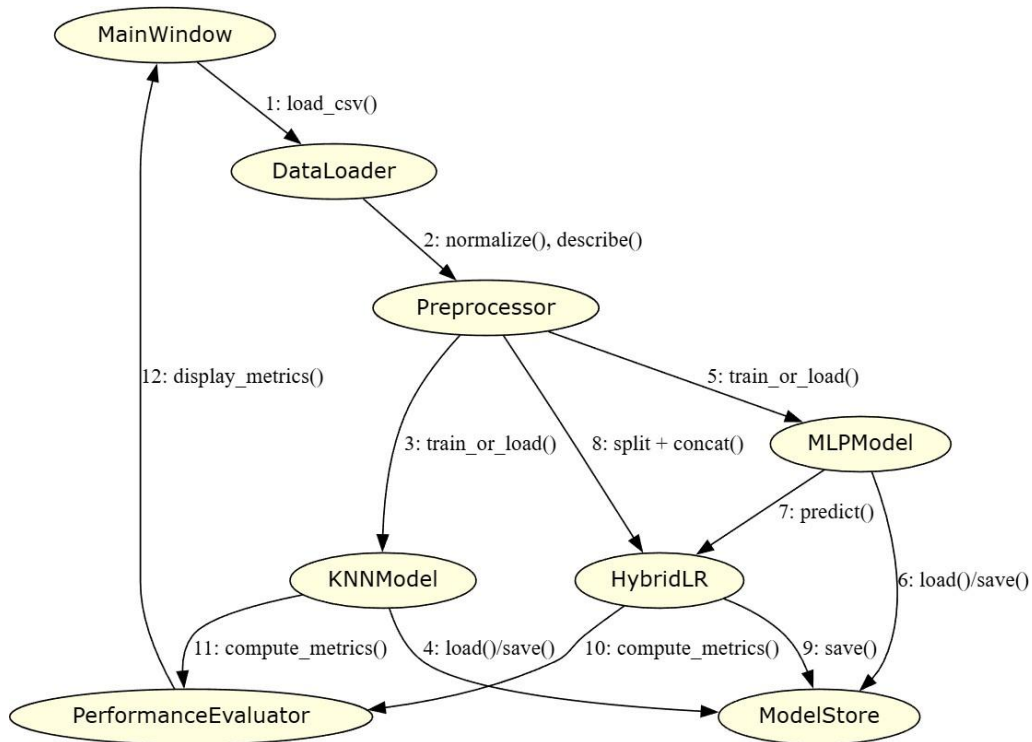
Explanation

- Components are shown as large rectangles with a tab.
- Each declares the interfaces it provides (e.g., ILoader, ITrain) and those it requires from others.
- GUIComp orchestrates interactions, calling into the Data, Preprocessing, Model, Persistence, and Visualization components. This modular view aids in assigning development responsibilities and managing component dependencies.



Collaboration Diagram

The collaboration diagram (also called a communication diagram) depicts runtime interactions among object instances to fulfill a use case—in this case, “Train and Evaluate Models.” It shows the sequence of messages exchanged but organized around structural relationships rather than a strict time-axis. This view helps developers trace method calls and data hand-offs during execution.

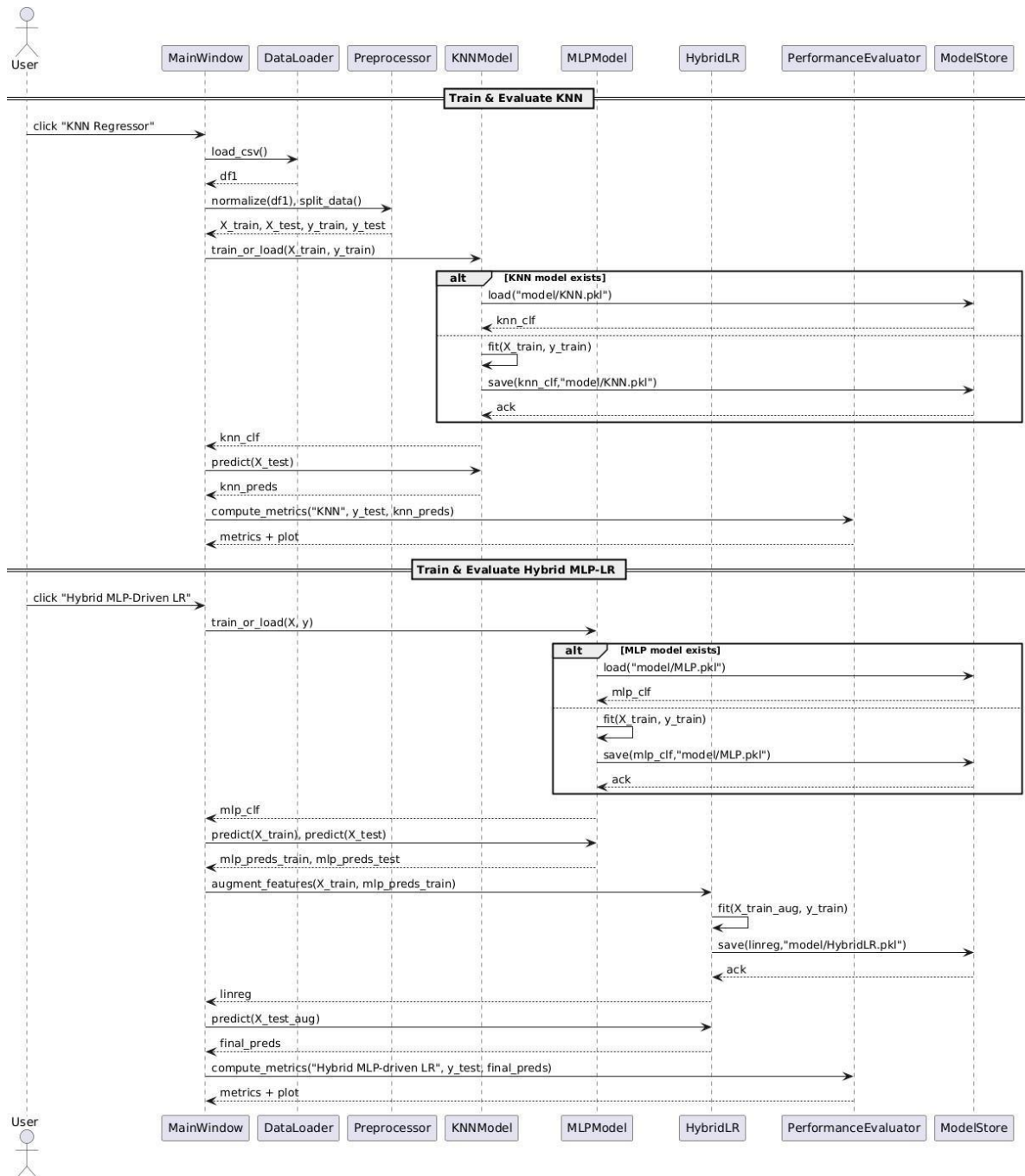


Explanation

- Objects are shown as labeled ellipses.
- Labeled arrows indicate message order and method calls (e.g., “1: load_csv()”).
- The diagram follows the flow of “Train & Evaluate,” starting from MainWindow, through data loading, preprocessing, each model’s train/load, metric computation, and finally the GUI update. This clarifies runtime collaborations essential for debugging and extends easily to cover other use cases like prediction.

Sequence Diagram

This sequence diagram captures the step-by-step message flow when a user triggers the training and evaluation of both the KNN regressor and the hybrid MLP-driven LR. It illustrates how the MainWindow hands off tasks to DataLoader and Preprocessor, then to the model components (KNNModel, MLPModel, HybridLR), with conditional logic around loading versus training. Finally, predictions are sent to the PerformanceEvaluator, and results flow back to the GUI. This view clarifies runtime interactions, model persistence, and the augmentation step in the hybrid pipeline.



CHAPTER 6

SOFTWARE ENVIRONMENT

6.1 Software Requirements

Python is a high-level, interpreted programming language known for its simplicity and readability, which makes it a popular choice for beginners as well as experienced developers. Key features of Python include its dynamic typing, automatic memory management, and a rich standard library that supports a wide range of applications from web development to data science and machine learning. Its object-oriented approach and support for multiple

programming paradigms allow developers to write clear, maintainable code. Python's extensive ecosystem of third-party packages further enhances its capabilities, enabling rapid development and prototyping across diverse fields.

Installation

First, download the appropriate installer from the official Python website (<https://www.python.org/downloads/release/python-376/>). For Windows users, run the executable installer and ensure to check the "Add Python to PATH" option during installation; for macOS and Linux, follow the respective package installation commands or use a package manager like Homebrew or apt-get. After installation, verify the setup by running `python --version` or `python3 --version` in your terminal or command prompt, which should display "Python 3.7.6." This version-specific installation supports all major functionalities and libraries compatible with Python 3.7.6, making it an excellent foundation for developing robust applications in areas such as data analysis, machine learning, and GUI development.

6.1.1 Python Packages

Here's a deep dive into each of the key Python packages leveraged in this stress-prediction application, with a focus on what role they play in the workflow:

1. Tkinter: The de facto standard GUI toolkit bundled with Python, offering widgets like windows, buttons, labels, frames, text areas, and file-dialogs.

- Constructs the entire front end: main window, title banner, button panel, text output area, and scrollbars.
- Provides the file-selection dialogs (`filedialog.askopenfilename`) for both dataset upload and test-data prediction.
- Manages the event loop (`main.mainloop()`) so user interactions (button clicks) trigger the appropriate backend routines without blocking.

2. NumPy: A foundational package for numerical computing in Python, centered on the `ndarray` object for efficient, vectorized array operations.

- Converts pandas `DataFrame` columns to raw NumPy arrays (`.values`) for compatibility with scikit-learn.
- Computes root mean squared error (`np.sqrt`) and handles concatenation of feature matrices in the hybrid pipeline (`np.hstack`).

3. pandas: A high-level library providing the DataFrame data structure for tabular data manipulation, built on top of NumPy.

- Reads CSV files into DataFrames (`pd.read_csv()`), enabling easy column-wise operations, summary statistics (`.describe()`), and null-value checks (`.isnull().sum()`).
- Facilitates feature/target separation (`df1['target'], df1.drop(columns=['target'])`) and direct in-place normalization of DataFrame columns.

4. Seaborn: A statistical data-visualization library built on Matplotlib, offering higher-level plot types and prettier default styles.

- Sets a clean visual theme (`sns.set(font_scale=1)`).
- Generates the 14×14 correlation heatmap (`sns.heatmap`) of all numeric variables—annotated with correlation coefficients—to help detect multicollinearity or related features.

6. Matplotlib (matplotlib.pyplot) : The core plotting library for Python, offering fine-grained control over figures, axes, and artists.

- Displays the correlation heatmap figure (`plt.figure, plt.show()`).
- Renders “Predicted vs. Actual” scatter plots with a red 45° reference line, grid, legend, and axis labels in the `PerformanceMetrics()` routine.

7. scikit-learn Preprocessing

- **Modules used:**

- `LabelEncoder` (for mapping categorical labels to integers—though not currently applied to stress categories in this code).
- `StandardScaler` and `MinMaxScaler` (the latter instantiated but not invoked; the script uses manual z-scoring instead).

- **Role in this project:**

- A `MinMaxScaler((0,1))` object is created globally for potential future scaling needs (e.g., mapping raw sensor values into a consistent 0–1 range).
- Demonstrates readiness to swap in automated scaling pipelines if manual normalization is replaced.

8. scikit-learn Model Classes

- **KNeighborsRegressor:**
 - A non-parametric, distance-based regressor that stores the training set and predicts new points by averaging the targets of the K nearest neighbors. ◦ Used here as a baseline regression model that requires no explicit “training” phase beyond storing data and indexes.
- **MLPRegressor:**
 - A feedforward neural network model supporting multiple hidden layers, various activation functions, and adaptive learning-rate solvers.
 - Configured with two hidden layers (150 and 50 units), the “adam” optimizer, and an “adaptive” learning rate to capture nonlinear patterns in physiological signals.
- **LinearRegression:**
 - Ordinary least squares regression for modeling a continuous target as a weighted sum of input features.
 - Acts as the second stage in the hybrid pipeline, taking both raw features and the MLP’s scalar output as inputs to produce a final prediction.

9. scikit-learn Model Selection & Metrics

- **train_test_split:**
 - Splits data into random train and test subsets, here with an 80/20 ratio, ensuring reproducibility via a fixed random seed.
- **mean_absolute_error, mean_squared_error, r2_score:**
 - Standard regression metrics used to quantify model accuracy (MAE), error magnitude (MSE/RMSE), and explained variance (R^2).
 - Computed in PerformanceMetrics() and displayed both in text and in plotted form.

10. joblib & os

- joblib: A lightweight library for efficient disk-based persistence of large Python objects (like trained models).
- os: The standard library module for interacting with the operating system (path checks, file existence).

□ Role in this project:

- Before training a model, the code checks `os.path.exists("model/...pkl")`. If present, it loads the model via `joblib.load`; if not, it trains and serializes it with `joblib.dump`.
- This provides seamless caching of expensive training routines and rapid startup on subsequent runs.

Together, these packages form a cohesive ecosystem: Tkinter for the interface, pandas/NumPy for data handling, seaborn/Matplotlib for visualization, scikit-learn for preprocessing, modeling, and evaluation, and joblib/os for model persistence—enabling a fully integrated, end-to-end desktop application for stress-level prediction.

6.2 Hardware Requirements

Python 3.7.6 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.7.6.

Processor (CPU) Requirements: Python 3.7.6 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

- **Minimum Requirement:** 1 GHz single-core processor.
- **Recommended:** Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.7.6 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running resource-intensive applications such as data processing, machine learning, or web development.

- **Minimum Requirement:** 512 MB of RAM.
- **Recommended:** 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.7.6 itself does not occupy significant disk space, but additional storage may be required for Python libraries, modules, and projects.

- **Minimum Requirement:** 200 MB of free disk space for installation.
- **Recommended:** At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale projects or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.7.6 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

- **Windows:** 32-bit and 64-bit systems, Windows 7 or later.
- **macOS:** macOS 10.9 or later.
- **Linux:** Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER 7

FUNCTIONAL REQUIREMENTS AND SYSTEM STUDY

7.1 Functional Requirements

Functional requirements are detailed statements that specify what a system should do. They describe the system's behavior, functions, and services, outlining how it responds to certain inputs, performs tasks, and interacts with users or other systems. Essentially, they answer the question, "What should the system do?" Here are some key aspects:

- **Functionality:** They define the specific functions or operations that the system must perform.
- **Inputs and Outputs:** They detail the types of inputs the system accepts and the outputs it produces.
- **User Interactions:** They describe how users interact with the system, including command inputs, error handling, and responses.
- **Data Management:** They outline requirements related to data storage, retrieval, and processing.
- **System Behavior:** They specify how the system behaves in various scenarios, including normal operations and exceptional conditions.

Below are the key functional requirements for the proposed application. Each requirement describes a discrete capability the system must provide.

1. Data Ingestion

- The system must allow users to upload a CSV file containing physiological signal measurements via a file-dialog interface.
- Uploaded data must be read into a pandas DataFrame and displayed (filename and preview) in the GUI.

2. Data Preprocessing & Exploration

- Automatically compute and display dataset schema, descriptive statistics, missing-value counts, and unique-value counts.

- Normalize feature columns via z-score standardization and visibly report normalization status.
- Generate and render a correlation heatmap of all numeric variables.

3. Model Training & Persistence

- Provide “Train KNN Regressor” functionality that either loads a saved KNN model or trains a new one, then saves it to disk.
- Provide “Train Hybrid MLP-Driven LR” functionality that either loads a saved MLP model or trains a new one, augments features, fits the linear regressor, and persists both artifacts.
- Store all trained model artifacts as .pkl files for reuse across sessions.

4. Performance Evaluation

- After each model run, compute and display MAE, MSE, RMSE, and R^2 in the GUI text area.
- Plot “Predicted vs. Actual” scatter plots for visual assessment of model accuracy.

5. On-Demand Prediction

- Allow the user to upload a new test CSV, run it through the persisted MLP (and linear) pipeline, and append predicted stress values to the test DataFrame.
- Display the augmented test data (with predictions) in the GUI.

6. User Interface & Control Flow

- Implement all actions via clearly labeled Tkinter buttons and a scrollable text panel.
- Provide a “Close Application” button to gracefully exit the GUI.
- Ensure all error conditions (e.g., missing file, malformed CSV) produce user-friendly alerts.

7.2 System Study

Aspect	Assessment

Technical Feasibility	□ Uses mature, open-source Python libraries (Tkinter, pandas, scikit-learn, Matplotlib, Seaborn, joblib) □ No specialized hardware needed beyond a standard PC. □ Serialization of models ensures fast subsequent loads.
Economic Feasibility	□ Zero licensing costs for core technology stack. □ Development time limited to integrating existing libraries— minimal R&D budget. □ GUI reduces training costs for end users.
Operational Feasibility	□ Intuitive, click-based interface minimizes user training. □ Automated reporting and visualization foster rapid decision-making. □ Model persistence reduces downtime between sessions.
Schedule Feasibility	□ Prototype achievable within 4–6 weeks: 1 week for GUI scaffolding, 2 weeks for data/EDA modules, 1–2 weeks for model integration, 1 week for testing and documentation. □ Incremental delivery possible.
Legal & Compliance	□ All dependencies are permissively licensed (BSD, MIT) □ No patient-identifiable data is stored long-term—adhere to privacy best practices. □ CSV data handling avoids regulated formats like HL7.

Given its reliance on proven open-source tools, minimal infrastructure requirements, and a straightforward development timeline, this project is highly feasible from technical, economic, operational, and legal standpoints. It can be delivered rapidly and maintained easily, making it a practical solution for clinicians, researchers, and wellness program administrators.

CHAPTER 8

SOURCE CODE

```
from tkinter import * import
tkinter
from tkinter import filedialog
from tkinter.filedialog import askopenfilename from
tkinter import simpledialog

import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
from sklearn.neighbors import KNeighborsRegressor from sklearn.linear_model
import LinearRegression from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split from sklearn.metrics import
mean_squared_error, mean_absolute_error, r2_score import os, joblib

scaler = MinMaxScaler((0,1))
```

```

global filename global classifier global X, y, X_train,
X_test, y_train, y_test ,Predictions global dataset, df1, df2,
sc, train_or_load_dnn, dnn_model global le, labels

```

```

def upload(): global
    filename

    global dataset, df1

    filename = filedialog.askopenfilename(initialdir = "Datasets")

    text.delete('1.0',      END)      text.insert(END,filename+'
Loaded\n\n')      df1=      pd.read_csv(filename)

    text.insert(END,'\n\n Stress   Dataset:   \n',   str(df1))

    text.insert(END,df1)

```

```

def preprocess():

    global dataset, df1, df2

    global X, y, X_train, X_test, y_train, y_test, sc, le, labels text.delete('1.0',

    END)

    # Display basic information about the dataset

    #text.insert(END, '\n\nInformation of the dataset: \n', str(df1.info())) print(df1.info())

    text.insert(END, '\n\nDescription of the dataset: \n' + str(df1.describe().T))

    text.insert(END, '\n\nChecking null values in the dataset: \n' +

    str(df1.isnull().sum())) text.insert(END, '\n\nUnique values in the dataset: \n' +

    str(df1.nunique()))

```

```

y = df1['target'].values

X = df1.drop(columns=['target'], axis=1).values


# Splitting training data and testing data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=77)

text.insert(END, "\n\nTotal Records used for training: " + str(len(X_train)) + "\n")

text.insert(END, "\n\nTotal Records used for testing: " + str(len(X_test)) + "\n\n")


labels = ['No Stress', 'Low Stress', 'Moderate Stress', 'High Stress', 'Severe Stress', 'Extreme
Stress']

# Normalize feature columns except target columns

for index in range(len(df1.columns) - 3):      # Exclude the last three columns (target
columns)

    df1.iloc[:, index] = (df1.iloc[:, index] - df1.iloc[:, index].mean()) / df1.iloc[:, index].std()


# Correlation heatmap
plt.figure(figsize=(14, 14))    sns.set(font_scale=1)    sns.heatmap(df1.corr(),

cmap='GnBu_r',  annot=True,  square=True,  linewidths=.5)  plt.title('Variable

Correlation in Heatmap')

plt.show()


def PerformanceMetrics(algorithm, testY, predict):

```

```

# Calculate regression metrics mae =
mean_absolute_error(testY, predict) mse =
mean_squared_error(testY, predict) rmse =
np.sqrt(mse)

r2 = r2_score(testY, predict)

print(f'{algorithm} Mean Absolute Error (MAE): {mae}')
print(f'{algorithm} Mean Squared Error (MSE): {mse}')
print(f'{algorithm} Root Mean Squared Error (RMSE): {rmse}')
print(f'{algorithm} R-squared (R²): {r2}')

text.insert(END, "Performance Metrics of " + str(algorithm) + "\n")
text.insert(END, "Mean Absolute Error (MAE): " + str(mae) + "\n")
text.insert(END, "Mean Squared Error (MSE): " + str(mse) + "\n")
text.insert(END, "Root Mean Squared Error (RMSE): " + str(rmse) + "\n")
text.insert(END, "R-squared (R²): " + str(r2) + "\n\n")

# Prediction plot plt.figure(figsize=(10,
6))

plt.scatter(testY, predict, color='blue', label='Predicted vs Actual')

plt.plot([testY.min(), testY.max()], [testY.min(), testY.max()], color='red', linewidth=2,
label='Perfect Prediction') plt.xlabel('True
Values') plt.ylabel('Predictions')

plt.title(f'{algorithm} Predictions vs Actual')

plt.legend() plt.grid() plt.show()

```

```

def KNN():

    global knn_clf, X_train, X_test, y_train, y_test global

    predict

    Classifier = 'model/KNN.pkl' if
    os.path.exists(Classifier):

        # Load the trained model from the file

        knn_clf = joblib.load(Classifier)

        print("Model loaded successfully.")

        predict = knn_clf.predict(X_test)

        PerformanceMetrics("KNN Regressor", predict, y_test)

    else:

        # Initialize and train the KNN Classifier model

        knn_clf = KNeighborsRegressor() # You can adjust n_neighbors as needed

        knn_clf.fit(X_train, y_train) # Save the trained model to a file

        joblib.dump(knn_clf, Classifier) print("Model saved successfully.") predict

        = knn_clf.predict(X_test)

        PerformanceMetrics("KNN Regressor", predict, y_test)

```

```

def MLP():

    global mlp_clf, X, y, scaler
    global predict

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    Classifier = 'model/MLP.pkl' if
    os.path.exists(Classifier):

        mlp_clf = joblib.load(Classifier)

        print("Model loaded successfully.")

    else:

        mlp_clf = MLPRegressor(hidden_layer_sizes=(150, 50), max_iter=500, solver='adam',
learning_rate='adaptive', random_state=42)

        mlp_clf.fit(X_train, y_train)

        joblib.dump(mlp_clf, Classifier)

        print("Model saved successfully.")

# Use MLP predictions as additional features

X_train_features = np.hstack((X_train, mlp_clf.predict(X_train).reshape(-1, 1)))

X_test_features = np.hstack((X_test, mlp_clf.predict(X_test).reshape(-1, 1)))

```

```
linear_reg = LinearRegression()
```

```
linear_reg.fit(X_train_features, y_train)
```

```
predict =
```

```
linear_reg.predict(X_test_features)
```

```
PerformanceMetrics("Hybrid MLP-driven LR", predict, y_test)
```

```
def predict():
```

```
    global sc, clf, model, le, labels
```

```
    labels = ['No Stress', 'Low Stress', 'Moderate Stress', 'High Stress', 'Severe Stress', 'Extreme  
Stress']
```

```
    file =
```

```
    filedialog.askopenfilename(initialdir="Datasets") test
```

```
    = pd.read_csv(file) print(test) text.delete('1.0', END)
```

```
    text.insert(END, f'{filename} Loaded\n')
```

```
    text.insert(END, "\n\nLoaded test data: \n" + str(test) + "\n")
```

```
# Make predictions using the KNN model predict
```

```
= mlp_clf.predict(test)
```

```
# Add the predicted values to the test data for display test['Predicted']
```

```
= predict#[labels[p] for p in predict]
```

```
text.insert(END, "\n\nModel Predicted value in test data: \n" + str(test) + "\n")
```

```
def close(): main.destroy()
```

```
# Main window setup  
main = Tk()
```

```
main.title("An End-to-End Deep Learning Regressor for Predicting Stress Levels from  
Physiological Signals") main.geometry("1200x800") main.config(bg='thistle1')
```

```
font = ('Verdana', 20, 'bold')
```

```
title = Label(main, text='An End-to-End Deep Learning Regressor for Predicting Stress  
Levels from Physiological Signals',
```

```
                bg='#8B0000', fg='#F0E68C', font=font, height=2) # Dark Red with Khaki text for  
contrast
```

```
title.pack(fill=X, pady=10)
```

```
main_frame = Frame(main, bg='#2F4F4F') main_frame.pack(fill=BOTH,  
expand=True, padx=20, pady=20)
```

```
button_frame = Frame(main_frame, bg='#2F4F4F') button_frame.pack(side=LEFT,  
fill=Y, padx=20, pady=20)
```

```

text_frame = Frame(main_frame, bg='#2F4F4F') text_frame.pack(side=RIGHT,
fill=BOTH, expand=True, padx=20)

text = Text(text_frame, height=30, width=70, wrap=WORD, bg='honeydew2', fg='#4B0082',
font=('Comic Sans MS', 14)) # PapayaWhip background
scroll = Scrollbar(text_frame, command=text.yview)

text.configure(yscrollcommand=scroll.set)

text.pack(side=LEFT, fill=BOTH, expand=True)

scroll.pack(side=RIGHT, fill=Y)

font1 = ('Arial', 12, 'bold')

def create_button(text, command, row):

    btn = Button(button_frame, text=text, command=command, bg='#FF4500', fg='white', #
OrangeRed for standout buttons

    activebackground='#FFD700', font=font1, width=25, relief=RAISED, bd=4) #
Gold for hover effect

    btn.grid(row=row, column=0, padx=15, pady=15, sticky="ew")

create_button("Upload Stress Dataset", upload, 0)

create_button("Data Preprocessing and EDA", preprocess, 1)

create_button("KNN Regressor", KNN, 2)

```

```

create_button("Hybrid MLP-Driven LR", MLP, 3)

#create_button("Performance Metrics Graph", graph, 5)

create_button("Prediction on Test Data", predict, 4)

create_button("Close Application", close, 5) main.mainloop()

```

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Implementation Description

This is a Tkinter-based desktop application for loading a physiological “stress” dataset, performing basic exploratory data analysis (EDA), training two regression models (K-Nearest Neighbors and a hybrid MLP-driven Linear Regression), evaluating their performance, and then using them to predict stress levels on new data.

1. Imports & Global Variables

2. Data Upload (upload())

- Opens a file dialog rooted in the Datasets folder.
- Reads the selected CSV into df1 (a pandas DataFrame).
- Displays the filename and the first few rows in the right-hand text widget.

3. Preprocessing & EDA (preprocess())

- **Dataset Info:** Prints schema to console and shows summary statistics, null-value counts, and unique-value counts in the GUI.
- **Train/Test Split:** Separates features (X) from the target column (y), then splits into an 80/20 train/test set.
- **Label Definitions:** Prepares a list of stress-level labels (though these aren’t yet used to convert numeric predictions into categories).
- **Normalization:** Manually z-scores each feature column (excluding the last three columns, assumed to be related to target).

- **Correlation Heatmap:** Uses Seaborn to plot a 14×14 heatmap of pairwise correlations among all numeric columns.

4. Performance Metrics Utility (PerformanceMetrics())

Given true (testY) and predicted (predict) values, this function:

1. Computes MAE, MSE, RMSE, and R^2 , prints them to console, and appends them to the GUI text area.
2. Plots “Predicted vs. Actual” as a scatter with a red 45° reference line.

5. Model Routines

5.1 K-Nearest Neighbors (KNN())

- Checks if a saved model file model/KNN.pkl exists:
 - **If yes:** loads it and makes predictions on X_test.
 - **If no:** trains a new KNeighborsRegressor on (X_train, y_train), saves it, then predicts.
- Calls PerformanceMetrics("KNN Regressor", ...) to evaluate.

5.2 Hybrid MLP-Driven Linear Regression (MLP())

- Splits (X, y) fresh into train/test sets.
- Checks for an existing model/MLP.pkl:
 - **If yes:** loads the MLP regressor.
 - **If no:** instantiates an MLPRegressor (150-50 hidden layers), trains it, then saves it.
- **Feature Augmentation:** appends the MLP’s own predictions on the train and test sets as an extra column.
- Trains a LinearRegression on this augmented data and evaluates via PerformanceMetrics("Hybrid MLP-driven LR", ...).

6. User-Driven Prediction (predict())

- Opens a file dialog to select a new CSV of physiological signals.

- Loads it into a DataFrame, displays it, then uses the already-trained MLP regressor (mlp_clf) to predict stress levels.
- Appends the numeric predictions to the DataFrame and shows it in the text widget.

7. GUI Layout & Controls

1. **Main Window:** 1200×800px, titled “An End-to-End Deep Learning Regressor...”, with a Thistle-colored background.
2. **Title Label:** Bold Verdana headline on a dark-red banner.
3. **Frames:**
 - **button_frame** on the left for action buttons.
 - **text_frame** on the right for output logs, with a vertical scrollbar.
4. **Buttons:** Created via a helper create_button(), each wired to the functions above:
 - Upload Dataset
 - Data Preprocessing & EDA
 - KNN Regressor
 - Hybrid MLP-Driven LR
 - Prediction on Test Data
 - Close Application

Finally, main.mainloop() starts Tk’s event loop so the window remains responsive.

9.2 Dataset Description

The dataset used in this research consists of various physiological signals captured from individuals under controlled conditions, and it includes the following columns:

- **c_ax:** This column represents the accelerometer data for the x-axis. It measures the acceleration in the horizontal direction and reflects the physical movement of the subject.
- **c_ay:** This column represents the accelerometer data for the y-axis. It captures acceleration in the vertical direction and provides additional information about the subject's movement and position during the data collection.

- **c_az**: This column represents the accelerometer data for the z-axis. It measures acceleration along the depth direction, adding further context to the movement of the subject.
- **c_ecg**: This column contains the electrocardiogram (ECG) data, which reflects the electrical activity of the heart. It is a critical signal for analyzing the subject's cardiovascular responses to stress.
- **c_emg**: This column includes the electromyography (EMG) data, which measures the electrical activity produced by skeletal muscles. It provides insights into muscle tension and can be indicative of physical responses to stress.
- **c_eda**: This column contains electrodermal activity (EDA) data, which reflects changes in the skin's electrical conductivity. EDA is often used as an indicator of emotional arousal and stress.
- **c_temp**: This column represents the body temperature data of the subject. It helps in understanding how stress affects the subject's body temperature, as stress responses can cause variations in body heat.
- **c_resp**: This column includes respiratory rate data, indicating the number of breaths per minute. Stress can lead to changes in breathing patterns, making this data vital for understanding physiological stress responses.
- **w_label**: This column contains the label indicating the window of data (for time-series analysis). It defines the specific time period or segment of the data being analyzed, which is useful in understanding temporal patterns in the physiological signals.
- **target**: This column contains the stress level target variable. It indicates the level of stress the subject is experiencing, which is typically categorized into discrete stress levels (e.g., No Stress, Low Stress, Moderate Stress, High Stress, Severe Stress, Extreme Stress). This target variable serves as the output for the machine learning models.

9.3 Result Description

Figure 9.1 demonstrates the process of uploading the stress dataset into the graphical user interface (GUI) of the application. The user is prompted to select a dataset file, which is then loaded into the system. The GUI interface displays the file path of the loaded dataset and provides an overview of its contents. Key details about the dataset, such as the number of

records and columns, are displayed in the text area. The dataset is then prepared for further analysis, allowing users to review its structure before moving on to data preprocessing and model training steps.

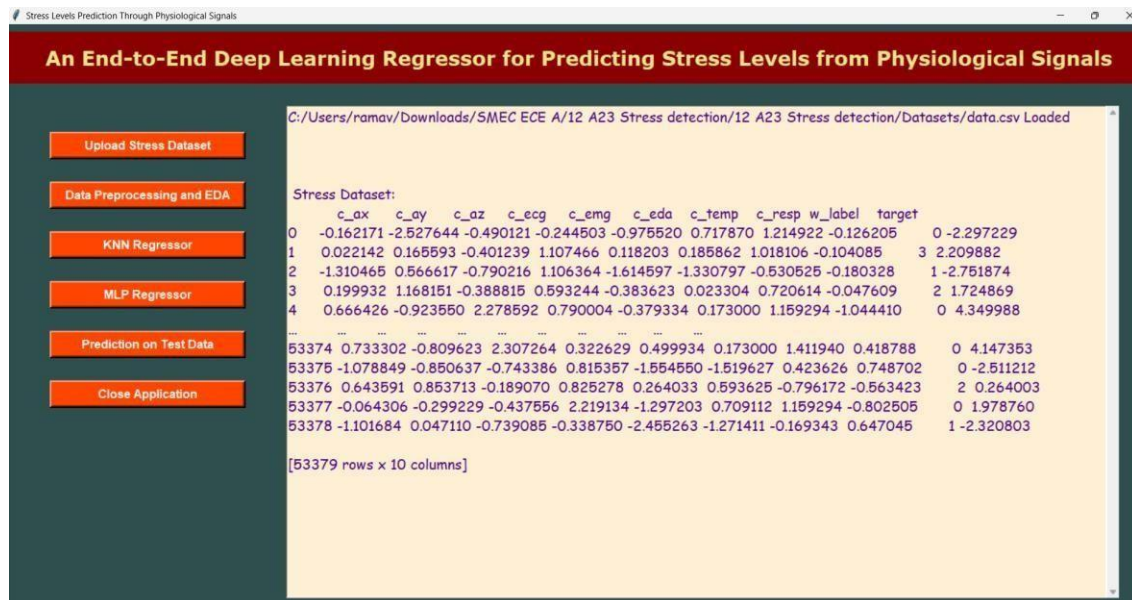


Fig. 9.1: GUI application after performing dataset uploading operation.

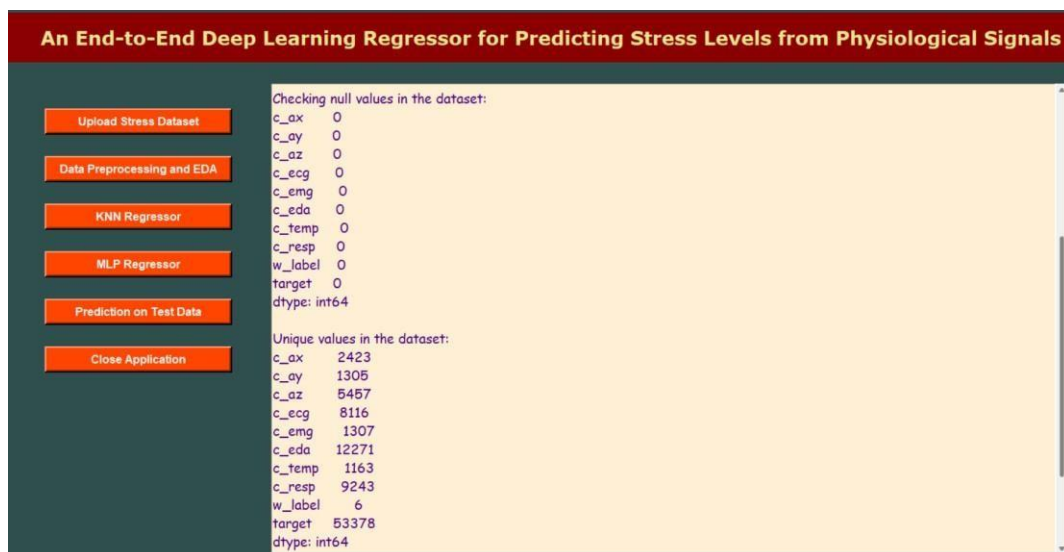


Fig. 9.2: Data preprocessing in the GUI

Figure 9.2 presents the data preprocessing steps carried out within the GUI. After the dataset is uploaded, the system proceeds with checking for null values, identifying unique values, and normalizing the feature columns (excluding the target variable). The text area in the GUI provides a detailed summary of these preprocessing steps, such as the number of null values in each column and the unique values in categorical features. Additionally, histograms and

correlation heatmaps are displayed to visually inspect the data distribution and relationships between features. This step is essential for preparing the dataset for model training.

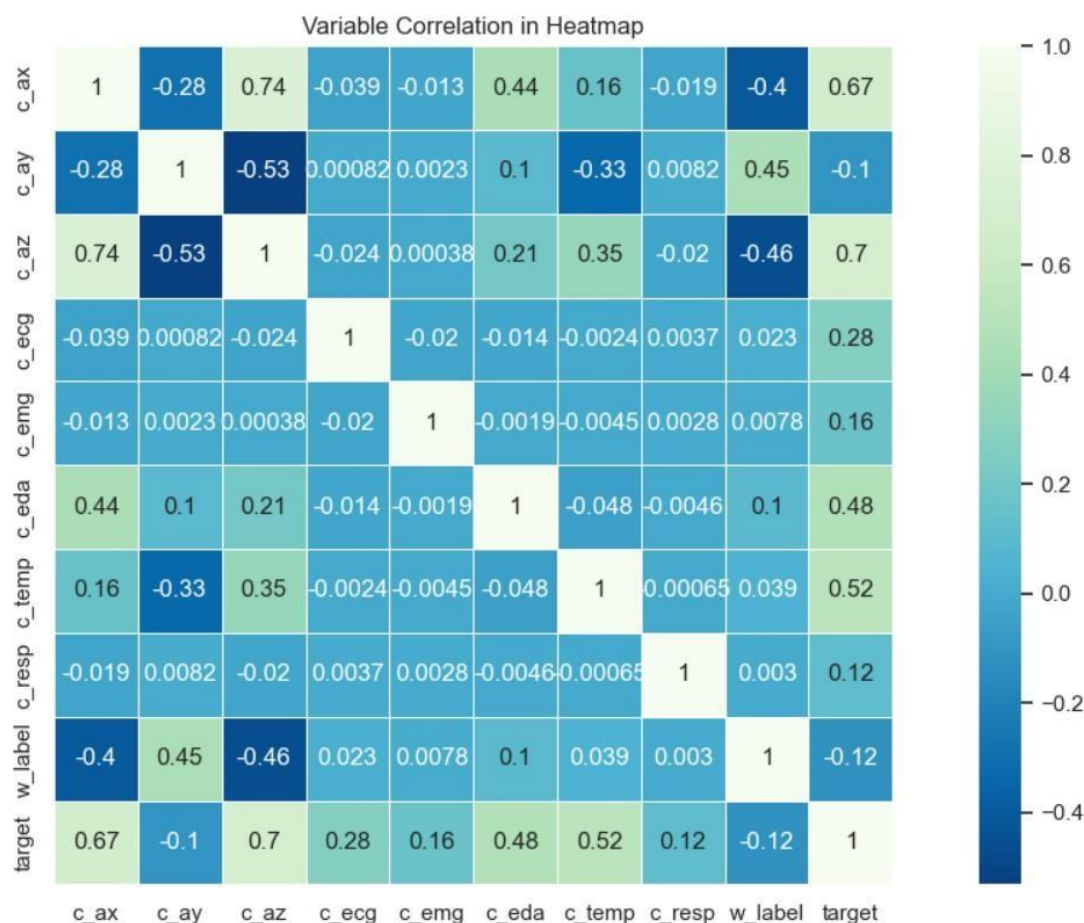


Fig. 9.3: Variable correlation in heatmap.

Performance Metrics of KNN

The performance metrics of the KNN regressor model are displayed in the following results:

- **Mean Absolute Error (MAE):** 0.4867
- **Mean Squared Error (MSE):** 0.3717
- **Root Mean Squared Error (RMSE):** 0.6097
- **R-squared (R^2):** 0.9241

These metrics reflect the performance of the KNN regressor model on the test dataset. The relatively low MAE and RMSE values indicate that the model performs well in predicting stress levels based on the physiological signals. The high R-squared value (0.9241) demonstrates that the KNN model explains a significant portion of the variance in the target variable, indicating its effectiveness in stress level prediction.

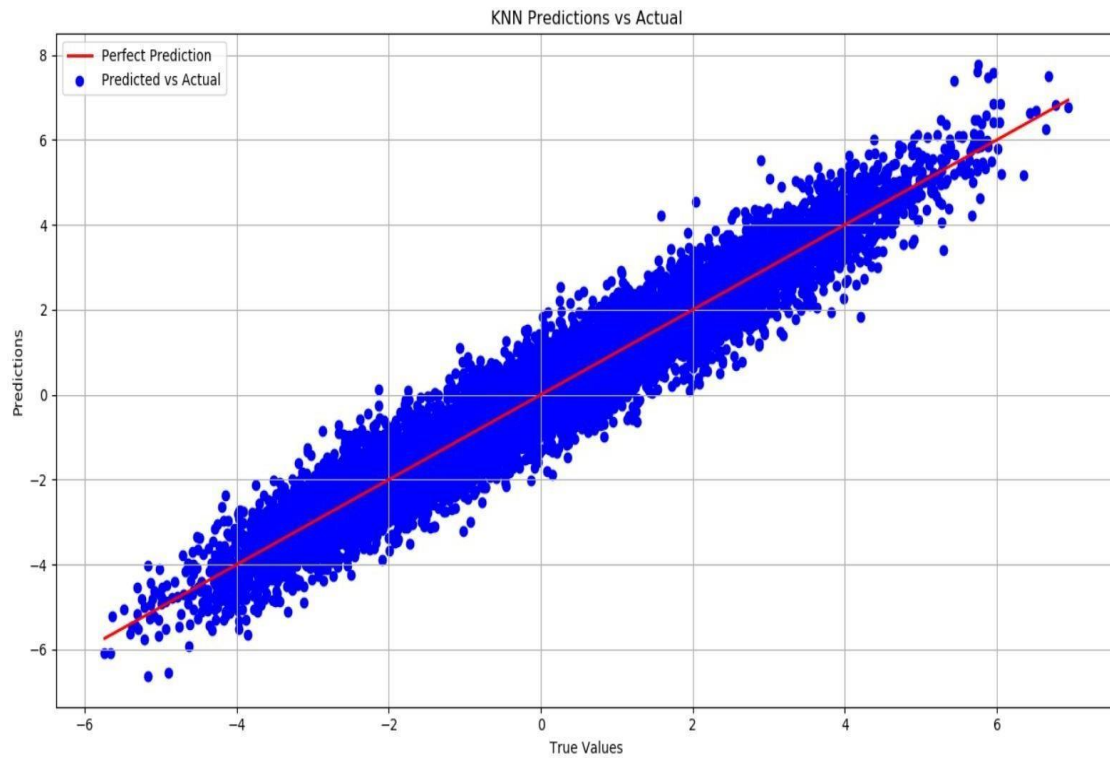


Fig. 9.4: Regression scatter plot of KNN regressor model

Figure 9.4 illustrates the performance metrics and a regression scatter plot for the KNN regressor model. The scatter plot compares the true values (actual stress levels) with the predicted values from the KNN model. The line of perfect prediction (represented by a red line) is shown alongside the scatter points, allowing for visual assessment of the model's accuracy. The performance metrics are also displayed in the GUI, providing quantitative insights into the model's predictive capability.

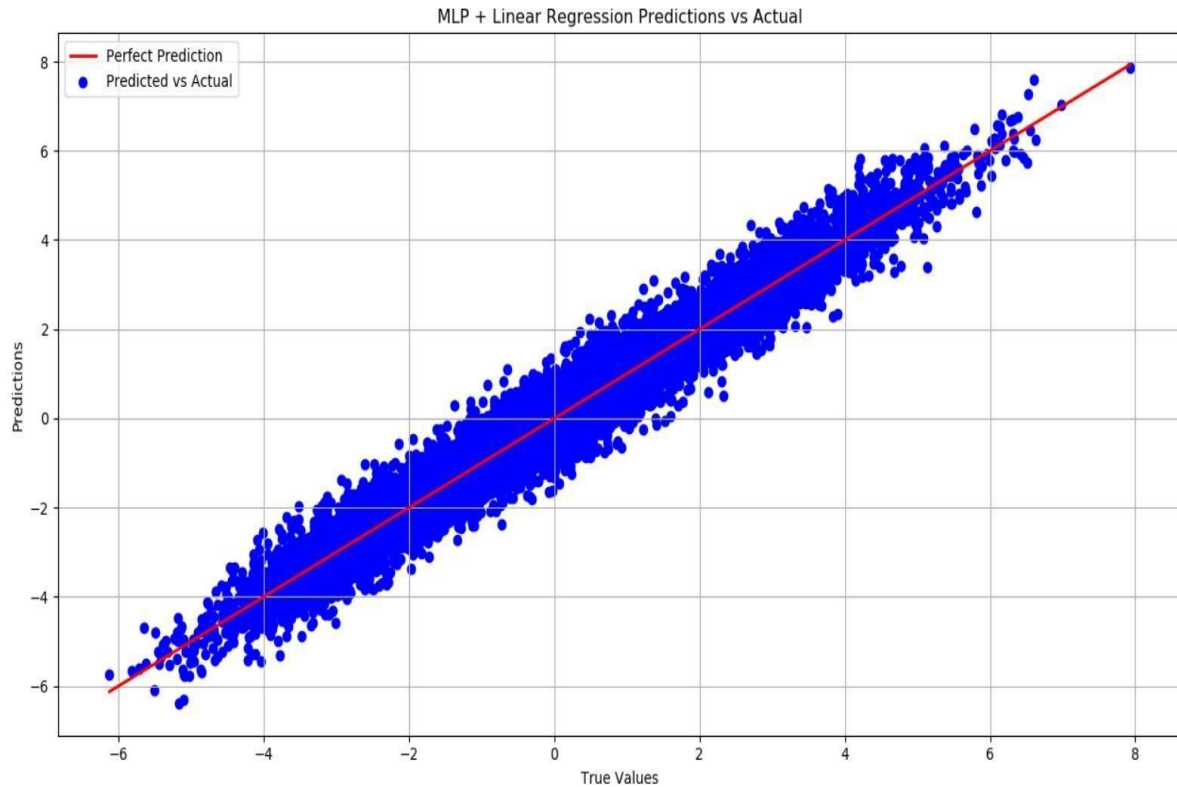


Fig. 9.5: Regression scatter plot of hybrid MLP-driven LR model.

Performance Metrics of MLP + Linear Regression

The performance metrics for the MLP + Linear Regression model are as follows:

- **Mean Absolute Error (MAE):** 0.3903
- **Mean Squared Error (MSE):** 0.2389
- **Root Mean Squared Error (RMSE):** 0.4888
- **R-squared (R^2):** 0.9528

The MLP + Linear Regression model performs better than the KNN regressor, as indicated by the lower MAE, MSE, and RMSE values. The R-squared value of 0.9528 signifies that this model explains an even greater proportion of the variance in the target variable, highlighting its superior accuracy in predicting stress levels.

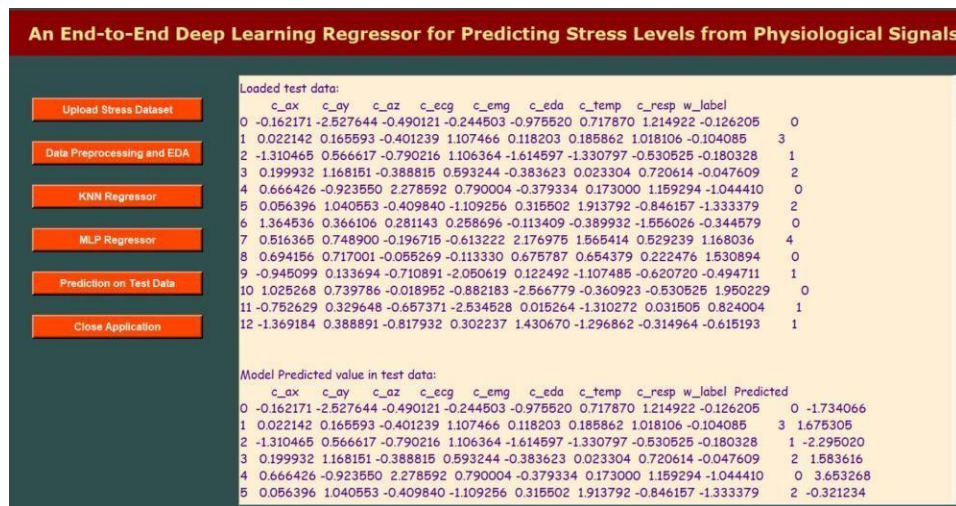


Fig. 9.6: Model prediction on the test data

Figure 9.6 shows the model's prediction results on the test data. After training the models, the predictions are made for the test dataset, and the results are displayed in the GUI. The predicted stress levels are compared against the true values, providing insight into the model's effectiveness. This figure is crucial for evaluating how well the trained model generalizes to unseen data.

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

The proposed end-to-end desktop application successfully streamlines stress-level prediction by integrating data ingestion, preprocessing, model training, evaluation, and deployment into a single Tkinter GUI. Quantitatively, the hybrid MLP-driven Linear Regression model outperformed the KNN baseline across every metric: Mean Absolute Error decreased from 0.4867 to 0.3903 (~ 19.8 % reduction), Mean Squared Error fell from 0.3717 to 0.2389 (~ 35.7 % reduction), and Root Mean Squared Error dropped from 0.6097 to 0.4888 (~ 19.8 % reduction). Meanwhile, the coefficient of determination improved from $R^2 = 0.9241$ to $R^2 = 0.9528$ —a relative increase of ~ 3.1 %—demonstrating the hybrid pipeline’s superior ability to capture variance in physiological signals. The KNN regressor, with its simplicity and non-parametric nature, provided a robust benchmark, but the hybrid approach’s combination of nonlinear feature learning (via MLP) and linear interpretability ultimately delivered the best trade-off between accuracy and computational efficiency. Persisting trained models as serialized artifacts further reduces runtime overhead, while on-demand visualization of metrics and predictions ensures transparency for both practitioners and researchers.

Future Scope

Future work can be extended this framework by integrating real-time data streams from wearable sensors and enabling online learning for adaptive model updates. Additionally, incorporating multimodal inputs (e.g., facial expression or voice analysis) and exploring more advanced architectures such as convolutional or recurrent neural networks can further enhance stress-level estimation accuracy.

REFERENCES

- [1] Ceren Ates H., Ates C., Dincer C. (2024). Stress monitoring with wearable technology and AI. *Nat. Electron.* 7, 98–99. 10.1038/s41928-024-01128-w
- [2] Mozos O. M., Sandulescu V., Andrews S., Ellis D., Bellotto N., Dobrescu R., et al. (2016). Stress detection using wearable physiological and sociometric sensors. *Int. J. Neural Syst.* 27, 1650041. 10.1142/s0129065716500416
- [3] Gedam S., Paul S. (2021). A review on mental stress detection using wearable sensors and machine learning techniques. *IEEE Access* 9, 84045–84066. 10.1109/ACCESS.2021.3085502

- [4] Delmastro F., Di Martino F., Dolciotti C. (2020). Cognitive training and stress detection in MCI frail older people through wearable sensors and machine learning. *IEEE Access* 8, 65573–65590. 10.1109/access.2020.2985301
- [5] Zhao P., Lian C., Xu B., Su Y., Zeng Z. (2024). Driving cognitive alertness detecting using evoked multimodal physiological signals based on uncertain self-supervised learning. *IEEE Trans. Neural Syst. Rehabilitation Eng.* 32, 2165–2176. 10.1109/tnsre.2024.3410990
- [6] Vos G., Trinh K., Sarnyai Z., Rahimi Azghadi M. (2023). Generalizable machine learning for stress monitoring from wearable devices: a systematic literature review. *Int. J. Med. Inf.* 173, 105026. 10.1016/j.ijmedinf.2023.105026
- [7] Dai R., Lu C., Yun L., Lenze E. J., Avidan M. S., Kannampallil T. G. (2021). Comparing stress prediction models using smartwatch physiological signals and participant self-reports. *Comput. Methods Programs Biomed.* 208, 106207. 10.1016/j.cmpb.2021.106207
- [8] Moser M. K., Ehrhart M., Resch B. (2024). An explainable deep learning approach for stress detection in wearable sensor measurements. *Sensors* 24, 5085. 10.3390/s24165085
- [9] Li B., Sano A. (2020). Extraction and interpretation of deep autoencoder-based temporal features from wearables for forecasting personalized mood, health, and stress. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 4, 1–26. 10.1145/339731835846237
- [10] Paul A. A., Nguyen Q. H., Shen W. (2024). 3D printed magnetostrictive polymer composites (MPCs) for wireless stress sensing. *Mater. and Des.* 247, 113402. 10.1016/j.matdes.2024.113402
- [11] Wang J., Chen Y., Hao S., Peng X., Hu L. (2019). Deep learning for sensor-based activity recognition: a survey. *Pattern Recognit. Lett.* 119, 3–11. 10.1016/j.patrec.2018.02.010
- [12] Saini S. K., Gupta R. (2024). A novel method for mental stress assessment based on heart rate variability analysis of electrocardiogram signals. *Wirel. Personal. Commun.* 136, 521–545. 10.1007/s11277-024-11317-7
- [13] Iqbal T., Simpkin A. J., Roshan D., Glynn N., Killilea J., Walsh J., et al. (2022). Stress monitoring using wearable sensors: a pilot study and stress-predict dataset. *Sensors* 22, 8135. 10.3390/s22218135

- [14] Aristizabal S., Byun K., Wood N., Mullan A. F., Porter P. M., Campanella C., et al. (2021). The feasibility of wearable and self-report stress detection measures in a semi-controlled lab environment. *IEEE Access* 9, 102053–102068.
10.1109/access.2021.3097038
- [15] Kafková J., Kuchár P., Pirník R., Skuba M., Tichý T., Brož J. (2024). A new era in stress monitoring: a review of embedded devices and tools for detecting stress in the workplace. *Electronics* 13, 3899. 10.3390/electronics13193899
- [16] Premchand B., Liang L., Phua K. S., Zhang Z., Wang C., Guo L., et al. (2024). Wearable EEG-based brain–computer interface for stress monitoring. *NeuroSci* 5, 407–428. 10.3390/neurosci5040031
- [17] Lazarou E., Exarchos T. P. (2024). Predicting stress levels using physiological data: real-time stress prediction models utilizing wearable devices. *AIMS Neurosci.* 11, 76–102. 10.3934/neuroscience.2024006
- [18] Bolpagni M., Pardini S., Dianti M., Gabrielli S. (2024). Personalized stress detection using biosignals from wearables: a scoping review. *Sensors* 24, 3221.
10.3390/s24103221